

The University of Melbourne

School of Mathematics and Statistics

Faculty of Science

MAST90107 Capstone Project Report

Data Format and Validation Registry

Abstract

This report presents the design, implementation, and evaluation of an integrated spatial data validation framework that systematically detects geometric, topological, and semantic errors across multiple types of geospatial datasets, including suburb boundaries, building footprints, commercial floor use blocks, and road networks. The objective is to enhance the integrity and interoperability of spatial data used in urban analytics, planning, and infrastructure management. The framework adopts a modular, rule-based architecture implemented in Python. Validation results are persistently stored in a lightweight relational registry built with SQLite. A web-based visualisation dashboard is implemented using Streamlit. It allows both technical and non-technical users to perform validation checks and view summarized quality reports.

Group 19

Members: Lei Cao, Xiaohan Deng, Xuanyi Zhang, Yaojin Wu, Zhizhou Zhu

Supervisor: Mitra Heidari

Affiliation: The University of Melbourne

Submission Date: October 2025



Faculty of Science
Melbourne, Australia

Declaration

I certify that this report does not incorporate, without acknowledgment, any material previously submitted for a degree or diploma in any university; and that, to the best of my knowledge and belief, it does not contain any material previously published or written by another person where due reference is not made in the text.

The report is [] words in length (excluding text in images, tables, bibliographies, and appendices).

Signed:

Lei Cao

Lei Cao

Xiaohan Deng

Xiaohan Deng

Xuanyi Zhang

Xuanyi Zhang

Yaojin Wu.

Yaojin Wu

Zhizhou Zhu

Zhizhou Zhu

Date: October 24, 2025

Acknowledgements

This project was conducted as a collaboration between the **University of Melbourne** and **AURIN**. We would like to express our sincere gratitude to our academic supervisor, **Mitra Heidari**, and our industry host, **Masoud Rahimi, Hao Chen, Bradley Greig**, for their valuable guidance, feedback, and support throughout the semester.

Contents

Declaration	1
Acknowledgements	2
1 Introduction	5
2 Related Works	6
3 Methodology	7
3.1 Suburb Boundary	7
3.1.1 Problem Formulation	8
3.1.2 Data and Preparation	8
3.1.3 Rules and Algorithms	9
3.1.4 Diagnostic Results	9
3.1.5 Evaluation Metrics and Performance	10
3.2 Statutory Block Boundary	10
3.2.1 Problem Formulation	11
3.2.2 Data and Preparation	11
3.2.3 Rules and Algorithms	11
3.2.4 Diagnostic Results	12
3.2.5 Evaluation Metrics and Performance	12
3.3 Building	13
3.3.1 Problem Formulation	13
3.3.2 Data and Preparation	13
3.3.3 Rules and Algorithms	14
3.3.4 Diagnostic Results	15
3.3.5 Evaluation Metrics and Performance	15
3.4 Commercial Floor Use Block	16
3.4.1 Problem Formulation	16
3.4.2 Data and Preparation	17
3.4.3 Rules and Algorithms	18
3.4.4 Diagnostic Results	18
3.4.5 Evaluation Metrics and Performance	19
3.5 Road Network	19
3.5.1 Problem Formulation	19
3.5.2 Data and Preparation	20
3.5.3 Rules and Algorithms	21
3.5.4 Diagnostic Results	21
3.5.5 Evaluation Metrics and Performance	21
4 Technical Framework	22
4.1 Technology Stack	22
4.1.1 Rule Engine Module: Python and GeoPandas	22
4.1.2 Validation Controller: Python-based Orchestration	22
4.1.3 Registry Storage: SQLite Database	23
4.1.4 Visualisation and Reporting: Streamlit	23
4.1.5 Extensibility and Reproducibility	23

4.1.6	System Architecture	23
5	Results	24
5.1	Registry	24
5.2	Dashboard	25
6	Discussion	27
6.1	Backend	27
6.2	Frontend	28
6.2.1	Functional advantage	28
6.2.2	Challenges	28
7	Conclusion and Future Improvements	28
7.1	Summary	28
7.2	Limitations and Future Improvements Suggestions	29
7.2.1	Automated Workflow Orchestration	29
7.2.2	Integration of Persistent Identifiers (PIDs)	29
7.2.3	Structural Consistency and Schema Adaptation	30
	Appendix: Repository and Meeting Records	31

1 Introduction

Data format and validation registry is an industry project supported by AURIN. We aim to build an extensible validation system for its diversified data sets.

Urban data infrastructure covers a wide range of complex data sets. However, data formats from different sources may be heterogeneous and lack unified standards, so it is difficult to find and correct data format and quality problems in time. The lack of a standardized verification framework is also detrimental to the integrity and interoperability of the data, making the data unable to fully meet the requirements of subsequent analysis and application.

Reliable spatial data are the basis of scientific decision-making and effective planning. Therefore, ensuring the consistency of data format and the accuracy of content is crucial for data analysis and scientific decision-making. Improving data quality through automated validation processes will significantly promote the improvement of urban infrastructure decision-making, optimization and robustness of resource allocation.

From a data-science perspective, end-to-end spatial validation and curation pose a tightly coupled set of challenges that extend well beyond simple geometry checks. First, volume and heterogeneity of formats are the norm rather than the exception: multi-gigabyte corpora arrive as GeoJSON, XML, CSV files, etc., often mixed with raster tiles and streamed API responses. Schemas evolve over time as fields are added, renamed, or deprecated. Robust ingestion therefore requires tolerant readers that reconcile schema drift, late-bind types, and handle partial or corrupt files without interrupting a batch run.

A second, pervasive issue is CRS heterogeneity and numeric precision. CRS refers to coordinate reference system, which is a framework used to precisely measure locations on the surface of the Earth as coordinates (Wikipedia contributors, 2025). The EPSG is a dataset as the standard for coordinate reference systems and transformations in the geospatial industry (Dataset, 2025). Inputs routinely mix geographic and projected CRSs (e.g., EPSG:4326 and EPSG:7855), or omit CRS metadata altogether. Because metric operations (area, length, buffers) are sensitive to projection choice and scale, the pipeline must assert or infer CRS, standardize to a project-appropriate CRS for measurements, and track rounding/quantization so that very small features do not collapse under finite precision. Subtle discrepancies in angular vs. metric units can otherwise masquerade as topological errors.

Third, modern layers exhibit complex spatial structure and dimensionality. Beyond simple polygons and lines, practitioners encounter MultiPolygons with nested holes, multipart buildings, near-duplicate vertices, spikes, bow-ties, and degenerate rings with zero area or length. Networks add node/edge topology and the requirement for shared nodes at crossings; in 3D, additional constraints arise around vertical consistency and non-self-intersecting shells. Ensuring logical consistency across these cases demands algorithms that are both numerically stable and explicit about tolerances.

Semantic diversity is equally significant. Attribute vocabularies vary across sources and jurisdictions—“locality”, “suburb”, and “block_id” may encode different concepts. Besides, units and valid ranges differ by context. Therefore, high-quality validation needs to check domain membership and unit conventions, enforce cross-field constraints (such as usage areas summing to a total). This aligns with external code lists or gazetteers when available and confirms temporal plausibility for date/time fields.

The result of the realities is multi-level rule complexity. Topology (overlaps, boundary alignment, gaps, network connectivity), effective validation spans geometry (validity, slivers, non-degeneracy), and semantics (nulls, duplicates) are all components of effective validation.

Rules must be composable and parameterized: overlap tolerances, boundary buffers, sliver ratios, and CRS policies should be adjustable per project or jurisdiction without changing core code. Since different error families interact—e.g., a projection mistake can inflate apparent overlaps—the system needs staged checks that report raw conditions before applying repairs like `buffer(0)`.

Finally, production use requires precise problem localisation, visualisation, repair, and storage. Diagnostics must identify the exact features and predicates that failed (who, where, why) and export map-ready subsets (e.g., GeoJSON) for rapid triage. Remediation workflows benefit from reversible diffs and clear metadata so that edits are auditable. A registry that persists dataset identifiers, rule profiles, timestamps, and per-rule metrics underpins reproducibility, supports side-by-side comparisons across runs, and enables simple dashboarding. At scale, performance considerations (R-tree indexing, tiling for parallelism), resilience (graceful handling of corrupt shapes or missing CRS), and governance (versioned rule profiles, provenance of thresholds) become first-class concerns. Addressing these challenges holistically is what turns a collection of checks into a reliable, extensible, and scientifically defensible spatial data quality pipeline.

To solve the above challenges, this project will design and implement an extensible automated data format and verification registration system. In this way, the format and quality verification of multi-source heterogeneous spatial data can be realised, and the overall quality and credibility of data can be finally improved.

2 Related Works

The consistency, accuracy, and reliability of spatial datasets are the focus of further study. With the increase in data complexity and volume, data validation methods have become more important. The key findings from the literature review are highlighted in data format and registry validation, and point out shortcomings and opportunities in current methods of spatial data quality assurance.

From the perspective of data validation, we reviewed *Validating 3D Land Parcels in Victoria* (Shojaei et al., 2017), which proposed validation guidelines for data integrity such as flat surfaces, volume consistency, no overlaps, and closed shapes. All these rules used geometric computations to increase data dependability and integrity. We also reviewed *Validating Massive Spatial Data with SpatialHadoop* (Song et al., 2012), which addressed guidelines to guarantee spatial consistency across geographic units of different scales and solved the challenge of validating massive spatial datasets using SpatialHadoop. These consist of the ComposedOf Rule, ExistIn rule, DjSiblings Rule. Another study called *A Structured Framework for 3D Cadastral Data Validation* (Asghari et al., 2020) created a framework to improve the quality and dependability of land administration systems by using rules for geometric validity, topological consistency, and semantic accuracy. *A graph-based algorithm to define urban topology from unstructured geospatial data* (de Almeida et al., 2013) introduced a graph-based model that represents spatial entities as nodes and edges, enforcing topological and attribute rules to preserve the integrity of geospatial data during processing.

Semantic-Driven Parametric 3D Geographic Scene Modelling (Dang et al., 2025) introduces a framework that combines Large Language Models with knowledge graphs to generate large 3D geographic scenes. It has a parametric modelling engine, RDF knowledge graphs and LLMs with chain-of-thought reasoning. The study *Application of Large Language Mod-*

els in Geographic Map Analysis and Visualisation (Nguyen et al., 2025) presents a GIS system utilising GPT-4o with a pipeline. It includes query-structure generation, automatic execution, solution-graph modelling, and empirical validation. By examining its design, interoperability, and performance, *Formalizing GeoJSON for Web-Based Spatial Data* (Butler et al., 2016) proposes using GeoJSON as a standard for web geospatial applications.

Validation Registries in SDIs from the data registry perspective highlights the significance of validation registries in Spatial Data Infrastructures (SDIs), demonstrating how registry architecture supports data quality with an Australian case. *A Review on Accuracy and Uncertainty of Spatial Data* (Devendran and Lakshmanan, 2014) outlines five essential quality dimensions: completeness, data lineage, attribute accuracy, positional accuracy, and logical consistency. Finally, *Geospatial Data Quality (ISO 19157-1): Evolve or Perish* (Ariza López et al., 2020) identifies five areas for improvement: expanding quality elements, improving quality measures, enhancing interoperability, supporting streaming data, and enabling machine-readable metadata.

According to Devendran and Lakshmanan (2014), geospatial datasets should be evaluated in relation to five key quality dimensions to ensure dependability and limit error propagation in GIS analyses: *lineage*, *logical consistency*, *positional accuracy*, *attribute accuracy*, and *completeness*. In this work we operationalise two of these dimensions. To ensure *logical consistency*, we apply topological rules to ensure that polygons are closed, lines are non-self-intersecting, and features do not overlap; in the road network scenario, this includes confirming that crossing roads are correctly noded rather than merely intersecting geometrically.

We quantify coverage and attribute completeness by examining null attributes, missing features, and spatial coverage gaps against the defined study area. Complementarily, Ariza López et al. (2020) highlights that geospatial quality standards such as ISO 19157 need to advance by adding new quality elements, enhancing interoperability with other standards through improved quality measures, supporting dynamic/real-time data, and adopting machine-readable metadata. Our automated diagnostics, rule-based and machine-readable outputs (CSV/GeoJSON) comply with these guidelines, facilitating reproducible and standards-aware quality evaluation.

3 Methodology

According to the literature review, there is no single and universal method for validating geospatial data. Different rules are required for different types of data. For instance, the dimensionality of geospatial data can differ; some datasets are two-dimensional while others are three-dimensional. Each format presents unique challenges and requires customized validation guidelines. The key is to think logically, such as asking whether a surface, soil, or property makes sense given its context.

3.1 Suburb Boundary

Suburb boundary datasets are typically represented as polygonal geometries with semantic attributes such as suburb name, postcode, and legal start date. Errors within such datasets compromise both spatial integrity and semantic consistency, leading to misinterpretation in policy planning or service allocation.

3.1.1 Problem Formulation

Formally, we define a dataset as:

$$D = \{(S_i, G_i, A_i) \mid i = 1, 2, \dots, n\}$$

where:

- S_i denotes the identifier (e.g., suburb name),
- G_i denotes the polygonal geometry of suburb i ,
- A_i denotes the set of semantic attributes (postcode, legal start date, etc.).

We detect and classify inconsistencies into three families:

$$\text{Geometric Errors} \quad E_g = \{i \mid \neg \text{is_valid}(G_i) \vee \text{area}(G_i) \leq 0\} \quad (1)$$

$$\text{Topological Errors} \quad E_t = \{(i, j) \mid i \neq j, \text{area}(G_i \cap G_j) > 0\} \cup \{i \mid \text{gaps/misaligned boundaries}\} \quad (2)$$

$$\text{Semantic Errors} \quad E_s = \{i \mid A_i \text{ contains nulls, duplicates, or invalid values}\} \quad (3)$$

Thus, the complete error set for dataset D is:

$$\mathcal{E}(D) = E_g \cup E_t \cup E_s.$$

3.1.2 Data and Preparation

We analyse a suburb boundary dataset showing the official suburb boundaries in South Australia as of August 2018 (Government of South Australia and Infrastructure, 2018). These boundaries defined under the Geographical Names Act 1991 for addressing purposes are based on cadastral, topographic, and road data. Features include **suburb**, **postcode**, **legalstartdate**, **shape_area**, and **shape_length**, etc. For metric operations, data are re-projected from geographic coordinates to a projected CRS in metres (e.g., EPSG:3107). Prior to checks, geometries undergo zero-width buffering to normalise simple invalidities, the layer is exploded where needed, and a spatial index (R-tree) is built to accelerate neighbourhood queries. The data are stored in a GeoJSON file, where a sample entry is like:

```
{
  "type": "Feature",
  "id": "sa_suburb_boundaries_aug_2018.1",
  "geometry": {
    "type": "MultiPolygon",
    "coordinates": [[[
      [138.53137231, -34.83256411],
      ...,
      [138.53137231, -34.83256411]
    ]]]
  },
  "geometry_name": "wkb_geometry",
  "properties": {
    "ogc_fid": 1,
```

```

    "postcode": "5013",
    "suburb": "GILLMAN",
    "suburb_number": 501301,
    "legalstartdate": "2009-10-21T13:00:00Z",
    "shape_length": 0.125751231186375,
    "shape_area": 3.83556221183474E-4
  }
},

```

which contains a MultiPolygon object with coordinates in latitude and longitude as well as the semantic features corresponding to this object. We then follow these pre-processing steps (1) Load into a GeoDataFrame; (2) fix simple invalids with `buffer(0)`; (3) reproject to EPSG:3107 for area/length; (4) run rules.

3.1.3 Rules and Algorithms

We implement a rule-based validator with modular functions:

- **Geometric validity:**
 - `check_geometries()` flags self-intersections/unclosed rings.
 - `check_areas()` ensures $\text{area}(G_i) > 0$ in metres.
- **Topological integrity:**
 - `check_overlaps()` returns pairs (i, j) with $\text{area}(G_i \cap G_j) > 0$.
 - `check_gaps()` contrasts dissolved vs. unioned coverage to expose voids/slivers.
 - `check_misaligned_boundaries()` probes adjacency alignment along shared borders.
- **Semantic consistency:**
 - `check_semantics()` detects duplicate suburb names, missing postcodes, and invalid `legalstartdate` entries via robust datetime coercion.

All rules are contained in a reusable Python class, enabling batched pipelines or single-dataset validation. Functions produce tabular diagnostics that are contained in a class **Failure**. This contains critical information for errors detected as well as geospatial layers that are suitable for visual inspection and remediation workflows where relevant. Outputs are exported as CSV and can be visualised in our lightweight web app to facilitate remediation.

3.1.4 Diagnostic Results

On the tested dataset: no invalid geometries and no non-positive areas after reprojection. Several overlapping suburb pairs were detected; no global topological gaps and no boundary misalignments beyond the 0.5 m tolerance. Multiple features contained invalid or missing `legalstartdate` values; suburb names and postcodes were otherwise consistent. These findings indicate quality debt concentrated in overlap resolution and attribute date cleaning.

3.1.5 Evaluation Metrics and Performance

We quantize the detection performance to evaluate the validator by defining *coverage* against a ground-truth or injected-error set E^* , following established spatial data quality assessment methodology (Devendran and Lakshmanan, 2014)

$$\text{Coverage} = \frac{|\mathcal{E}(D) \cap E^*|}{|E^*|}. \quad (4)$$

Pertinently, we modify the dataset with some toy errors and report the coverage, where the following ground truth errors are intentionally designed: Gamma is purposely self-intersecting (invalid polygon); Zeta is a degenerate polygon with area = 0; Alpha appears twice and the second Alpha has postcode = null; Delta carries an invalid date string; Alpha and Beta overlap with each other. Key features are shown in table 1,

Table 1: Toy suburb boundary dataset

suburb	postcode	legalstartdate	geometry
Alpha	3000	2018-08-01	POLYGON((0 0, 2 0, 2 2, 0 2, 0 0))
Beta	3001	2018-08-01	POLYGON((1 1, 3 1, 3 3, 1 3, 1 1))
Gamma	3002	2019-05-15	POLYGON((4 0, 6 2, 4 2, 6 0, 4 0))
Alpha	null	2020-01-20	POLYGON((7 0, 9 0, 9 2, 7 2, 7 0))
Delta	3004	2025-13-40	POLYGON((10 0, 12 0, 12 2, 10 2, 10 0))
Epsilon	3005	2017-03-10	POLYGON((0 3, 0.02 3, 0.02 3.02, 0 3.02, 0 3))
Zeta	3006	2016-07-01	POLYGON((13 0, 14 1, 15 2, 13 0))

As summarised in table 2, all types of errors are fully detected and the suburb rule checker reaches a coverage of 1. However, though the usability of this tool has been proven, the experiment is simple and not generalised, which may not reveal any underlying shortcomings and biases of the checker tool.

Table 2: Ground truth vs. reported detections on toy suburb dataset

Error type	Ground truth count	Reported count	Coverage
Geometric invalid	2	2	
Non-positive area	1	1	
Overlaps	1	1	
Duplicate suburb names	1	1	
Missing postcode	1	1	
Invalid dates	1	1	
Total	7	7	1.0

3.2 Statutory Block Boundary

We use legally defined *block* boundaries (statutory parcels/blocks) as an example of district-scale polygon datasets. Each record is a polygon or multipolygon with attributes describing the block identifier and related metadata. Geometry- or topology-level defects (self-intersection, non-simple rings, pairwise overlaps, non-manifold shared edges), as well

as shape-quality issues (extremely short exterior edges, acute angles, wrong exterior orientation) may cause downstream confusion in entitlement calculation, frontage/length measures, and adjacency-based analysis.

3.2.1 Problem Formulation

Let $B = \{(S_i, G_i, A_i) \mid i = 1, 2, \dots, n\}$, where S_i is a stable block ID, G_i a polygon, and A_i attributes. We define three error families:

$$\text{Geometric Errors} \quad E_g^{(B)} = \{i : \neg \text{valid}(G_i) \vee \neg \text{simple}(G_i)\}, \quad (5)$$

$$\text{Topological Errors} \quad E_t^{(B)} = \{(i, j) : i \neq j, \text{area}(G_i \cap G_j) > 0\} \cup \{i : \text{nonman}(G_i)\}, \quad (6)$$

$$\text{Semantic Errors} \quad E_s^{(B)} = \{i : \text{short}(G_i) \vee \text{acute}(G_i) \vee \text{orient}(G_i) \vee \text{prec}(G_i) \vee \text{tri}(G_i) \vee \text{oob}(G_i)\}. \quad (7)$$

The complete set is $\mathcal{E}(B) = E_g^{(B)} \cup E_t^{(B)} \cup E_s^{(B)}$.

Legend. valid: Shapely validity; simple: ring simplicity; nonman: non-manifold exterior edges; short: shortest exterior edge < 2 m; acute: min exterior angle $< 5^\circ$; orient: outer ring not CCW; prec: > 3 decimals on $> 10\%$ exterior vertices (projected CRS); tri: triangulation area gap $> 10^{-1}$; oob: outside bounding box $((140, -40), (150, -33))$.

3.2.2 Data and Preparation

Input formats may be GeoJSON/CSV/XML/KML; geometries are Polygon/MultiPolygon. Pre-processing follows a light normalisation pipeline: (1) fix simple invalids using zero-width buffering and explode multipart features; (2) build an R-tree spatial index; (3) project on demand to a metric CRS (EPSG:3107) for length-based checks. This mirrors the helper utilities in our implementation (safe CRS assignment, explode, STRtree).¹ *Default tolerances used below* come from the code: minimum exterior edge length = 2.0 m, minimum interior angle = 5° , maximum stored decimals = 3, triangulation area tolerance = 10^{-1} , and a sanity bounding box (140, -40)–(150, -33). :contentReference[oaicite:1]index=1

3.2.3 Rules and Algorithms

We implement a modular **BlockChecker** and a convenience runner `run_blocks_checks(gdf, min_len_m=2.0)` that aggregates failures into a tabular report. Main rules are:

- **Geometry validity / simplicity**
 - `check_invalid_geometry()`: flag invalid geometries.
 - `check_not_simple()`: flag non-simple (self-touching/self-crossing) rings.
- **Pairwise overlaps**
 - `check_overlaps(min_area=0.0)`: report each overlapping pair where intersection area > 0 (no tolerance), with per-side overlap area/ratio.
- **Shape quality (outer ring only)**

¹Implementation details follow our **BlockChecker** utilities for cleaning, projection, and spatial queries.

- `check_short_edges(min_len=2.0m)`: minimum exterior segment length must be ≥ 2 m (measured in EPSG:3107).
- `check_wrong_orientation()`: exterior ring expected CCW.
- `check_nonmanifold_edges()`: any undirected exterior edge shared by > 2 polygons is non-manifold.
- `check_min_angle(min_deg=5.0, include_interiors=false)`: minimum exterior angle must be $\geq 5^\circ$.
- `check_excessive_precision(max_decimals=3)`: in projected CRS, if >3 decimals occur at $>10\%$ exterior vertices, flag as excessive precision.
- `check_triangulation(tol=10-1)`: if the area difference between polygon and its triangulation exceeds `tol`, flag as not closed/numerically unstable.
- `check_outside_bbox(bbox=((140,-40),(150,-33)))`: flag geometries outside the given region.

3.2.4 Diagnostic Results

On the tested statutory block layer, the checker flagged geometry issues with 0 invalid features and 0 not-simple geometries. Topologically, it found 0 overlapping pairs (area > 0) and 0 features exhibiting non-manifold exterior edges. In terms of shape quality, it reported 0 features with a shortest exterior edge < 2 m, 0 with clockwise outer rings, 4 with minimum angle $< 5^\circ$, 0 with > 3 decimals at $> 10\%$ of exterior vertices, 3 failing triangulation tolerance, and 0 lying outside the bounding box.

Table 3: Reported detections by error type on the block dataset.

Error type	Count	Notes
InvalidGeometry	0	Shapely validity failed
NotSimple	0	Self-touching / self-crossing
Overlap (pairs)	0	Intersection area > 0
NonManifoldEdge	0	Exterior edge shared by > 2 polygons
ShortEdge	0	Shortest exterior edge < 2 m
WrongOrientation	0	Outer ring clockwise (expect CCW)
AcuteAngle	4	Min exterior angle $< 5^\circ$
ExcessivePrecision	0	>3 decimals at $> 10\%$ vertices
TriangulationNotClosed	3	Triangulated vs. original area diff $> 10^{-1}$
OutsideBBBox	0	Outside $((140,-40),(150,-33))$

3.2.5 Evaluation Metrics and Performance

Synthetic cases include: invalid and non-simple polygons, pairs with positive-area overlap, non-manifold sharing of an exterior edge, a polygon with an exterior edge < 2 m, a clockwise outer ring, minimum exterior angle $< 5^\circ$, excessive coordinate decimals, a triangulation area gap $> 10^{-1}$, and an outside-bbox geometry. The checker identifies each class under default thresholds, and STRtree-based neighbourhood screening keeps pairwise checks tractable on large layers.

3.3 Building

Building footprint datasets are polygonal (or multipolygonal) representations of individual buildings, optionally enriched with semantic attributes such as building identifiers and height. Typical issues include geometry-level defects (invalid rings, zero/near-zero area), topological conflicts (overlaps, self-overlaps among multipart buildings, sliver footprints), and semantic inconsistencies (duplicate identifiers, unexpected CRS).

3.3.1 Problem Formulation

Formally, we define the building dataset as follows:

$$B = \{(S_i, G_i, A_i) \mid i = 1, 2, \dots, n\},$$

where:

- S_i denotes the identifier (e.g., a stable building ID),
- G_i represents the polygonal footprint geometry of building i , and
- A_i denotes optional semantic attributes (height, use, roof form, etc.).

We categorize detected inconsistencies into three families:

$$\text{Geometric Errors} \quad E_g^{(B)} = \{i \mid \neg \text{is_valid}(G_i) \vee \text{area}(G_i) \leq 0\}, \quad (8)$$

$$\text{Topological Errors} \quad E_t^{(B)} = \{(i, j) \mid i \neq j, \text{area}(G_i \cap G_j) > \tau_{ov}\} \cup \{i \mid \text{parts in } G_i \text{ overlap or are sliver-like}\}, \quad (9)$$

$$\text{Semantic Errors} \quad E_s^{(B)} = \{i \mid \text{CRS not in allowed set} \vee \text{duplicate geometry/id} \vee \text{required attributes null}\}. \quad (10)$$

Thus, the complete error set for dataset B is:

$$\mathcal{E}(B) = E_g^{(B)} \cup E_t^{(B)} \cup E_s^{(B)}.$$

3.3.2 Data and Preparation

We have a GeoJSON dataset of buildings in Melbourne. Each point contains a building’s boundary shape `Polygon` or `MultiPolygon` possibly with its building number and height. These points were originally recorded using latitude and longitude (EPSG:4326 or EPSG:7844). While this is fine for viewing, calculating areas, distances, and overlaps in meters is inaccurate using latitude and longitude. Therefore, before performing these calculations, we convert the data to EPSG:7855, a projection coordinate system appropriate for the region, using meters. This ensures that the area and distance measurements are reliable.

Pre-processing replicates the suburb workflow. First, we load and clean the layer by reading it into a `GeoDataFrame`, applying `buffer(0)` to fix common polygon defects such as small self-intersections or tiny gaps, and explode any `MultiPolygon` into single parts for per-block verification. Next, we standardize the metric CRS by reprojecting to EPSG:7855 so areas and lengths are computed in metres; if the input CRS is absent or incompatible, we fall back safely to avoid stopping execution. Finally, we build a spatial index with `STRtree`

to accelerate neighbourhood queries and prune intersection candidates, which substantially reduces the cost of rules such as overlap and contact detection.

We expose a convenience runner `run_building_checks(...)` wiring standard thresholds: overlap tolerance $\tau_{ov} = 0.05 \text{ m}^2$, hole ratio ≤ 0.20 , total hole area $\leq 2.0 \text{ m}^2$, minimum area $\geq 1.0 \text{ m}^2$, and sliver ratio (area/perimeter) ≥ 0.20 .

3.3.3 Rules and Algorithms

All building validation functionality is implemented using a common `BuildingChecker` class (with a working CRS set to EPSG:7855). After execution, each rule generates a structured record containing an index, error category, information, and contextual data. The results are consolidated into a unified `DataFrame` for result aggregation, visualisation, and subsequent repair tracking. Main rules:

- **Geometric validity:**
 - `check_geometry_not_empty()` flags empty/NA geometries.
 - `check_geometry_valid()` attaches `explain_validity` reasons for invalid polygons.
 - `check_building_polygons_only()` building geometries must be `Polygon/MultiPolygon`.
 - `check_minimum_area(min_area = 1.0 m2)` rejects degenerate footprints in metric CRS.
 - `check_sliver_polygons(min_area_perimeter_ratio = 0.20)` — flags sliver-like shapes by area/perimeter.
- **Topological integrity:**
 - `check_overlapping_buildings(area_tolerance = 0.05 m2)` — disallows overlaps beyond a small tolerance; uses `STRtree (query_bulk)` and exact intersections to accumulate total overlap area per feature.
 - `check_parts_self_overlap()` — within a `MultiPolygon`, parts must not overlap (intra-building consistency).
- **Semantic consistency:**
 - `check_semantics()` detects duplicate suburb names, missing postcodes, and invalid `legalstartdate` entries via robust datetime coercion.
- **Interior cavities:**
 - `check_holes_area_ratio_limit(max_ratio = 0.20)` — sum of hole areas $\leq 20\%$ of footprint area.
 - `check_holes_total_area_limit(max_total_hole_area = 2.0 m2)` — absolute cap on total hole area.
- **CRS / duplication / attributes:**
 - `check_expected_crs(allowed={4326,7844})` — input CRS must be in the allowed set.
 - `check_duplicate_geometries()` — WKB-based duplicate geometry detection.
 - *Optional attribute rules:* `check_attribute_not_null(...)`, `check_unique_field(...)`, `check_attribute_in_domain(...)` when identifiers/heights/uses are available.

3.3.4 Diagnostic Results

After running validation on the analysed building dataset, the system organizes the detected results into a structured table (including an index, error type, description, and additional information). This table can be exported to CSV or viewed in a visualisation interface. In this analysis, most building features passed the geometry and area checks, with only two types of issues identified:

1. **HolesAreaRatioLimit (8 total)**: Some buildings exceeded the set limit (20%) for the ratio of internal void area, primarily in buildings with atria or courtyards.
2. **ExpectedCRS (1 total)**: A feature used a coordinate reference system that was not within the allowed range (not EPSG:4326, 7844, 7855, or 28355).

Table 4: Reported detections by error type on the building dataset.

Error type	Count	Notes
GeometryValid / Empty	0	Invalid or empty geometries
MinimumArea	0	Area $< 1.0 \text{ m}^2$
OverlappingBuildings	0	$\text{area}_{\text{overlap}} > 0.05 \text{ m}^2$
PartsSelfOverlap	0	Multipart parts overlap
SliverPolygons	0	Area/perimeter < 0.20
HolesAreaRatioLimit	8	Holes/area > 0.20
HolesTotalAreaLimit	0	Total holes $> 2.0 \text{ m}^2$
ExpectedCRS	1	CRS not in $\{4326, 7844, 7855, 28355\}$
DuplicateGeometries	0	Identical WKB

3.3.5 Evaluation Metrics and Performance

For demonstration, we curated a compact toy subset with seven deliberately injected defects: (i) a self-intersecting polygon, (ii) a zero-area/degenerate ring, (iii) two buildings that overlap with $\text{area}(G_i \cap G_j) > 0.05 \text{ m}^2$, (iv) a multipart footprint exhibiting self-overlap, (v) a sliver footprint where $\text{area}/\text{perimeter} < 0.20$, (vi) a polygon whose total hole area exceeds 2.0 m^2 , and (vii) a duplicate geometry. This minimal set exercises geometric, topological, and semantic rules in a controlled, fast-to-run scenario.

The checker identifies each class by the corresponding rule above. We summarise results in a table mirroring the suburb experiment:

Table 5: Ground truth vs. reported detections on toy building subset.

Error type	Ground truth	Reported
Geometric invalid	1	1
Minimum area	1	1
Overlaps	2	2
Parts self-overlap	1	1
Sliver polygons	1	1
Holes area ratio	1	1
Holes total area	1	1
Duplicate geometries	2	2
Total	10	10

3.4 Commercial Floor Use Block

Commercial floor-use block datasets record the use of different kinds of commercial spaces and display portions of a city by dividing into individual blocks. Typically, they include polygon forms with numerous descriptive fields that distinguish the purpose of each area. Verifying and updating this data is crucial for business research, resource sharing, and city planning (Bibri, 2020).

3.4.1 Problem Formulation

Formally, we define a commercial floor use dataset as:

$$C = \{(B_i, G_i, A_i) \mid i = 1, 2, \dots, n\}$$

where:

- B_i denotes the block identifier,
- G_i denotes the polygonal geometry of block i ,
- A_i denotes the comprehensive set of semantic attributes including various space usage types (commercial accommodation, office space, retail, educational facilities, etc.).

We categorise inconsistencies into three primary families:

$$\text{Geometric Errors} \quad E_g = \{i \mid \neg \text{is_valid}(G_i) \vee \text{area}(G_i) \leq 0\} \quad (11)$$

$$\text{Topological Errors} \quad E_t = \{(i, j) \mid i \neq j, \text{area}(G_i \cap G_j) > 0\} \quad (12)$$

$$\text{Semantic Errors} \quad E_s = \{i \mid A_i \text{ contains nulls, outliers, or inconsistent values}\} \quad (13)$$

The complete error set for data set C is:

$$\mathcal{E}(C) = E_g \cup E_t \cup E_s.$$

3.4.2 Data and Preparation

We analyze the 2011 Victorian Local Government Commercial Floor Use Block data set (City of Melbourne, 2011), which offers comprehensive details on the use of commercial space in Melbourne. The dataset includes 43 attributes with many categories such as:

- `commercial_accommodation`: Commercial accommodation space area
- `office`: Office space area
- `retail_shop`: Retail shop area
- `residential_apartment`: Residential apartment space area
- `educational_research`: Educational and research facilities
- `total_space_in_block`: Total available space in the block

With Australian spatial data standards, the data set uses the geographic coordinate system EPSG:4283 (GDA94) (Intergovernmental Committee on Surveying and Mapping (ICSM), 2020). For spatial operations requiring metric measurements, we preserve the original CRS due to the reason that the data set primarily serves analytical rather than precise geometric calculations. The data format is illustrated with an example entry taken from the GeoJSON file:

```
{
  "id": "com_clue_floor_use_block_2011.662",
  "block_id": 662,
  "census_year": 2011,
  "clue_small_area": "East Melbourne",
  "commercial_accommodation": 0.0,
  "office": 0.0,
  "retail_shop": 0.0,
  "residential_apartment": 0.0,
  "educational_research": 0.0,
  "total_space_in_block": 12542.0,
  "geometry": {
    "type": "MultiPolygon",
    "coordinates": [[[ [144.98996, -37.81761], [144.98881, -37.81753] ]]]
  }
}
```

Pre-processing steps include: 1. Utilise GeoPandas to import the GeoJSON file into a GeoDataFrame (Jordahl, 2020). 2. Verify and fix invalid geometries using `buffer(0)` method based on computational geometry principles (Shojaei et al., 2017). 3. Build an efficient spatial query by the R-tree spatial index (Song et al., 2012). 4. Evaluate semantic attributes for data quality assessment following ISO 19157 standards (Ariza López et al., 2020).

3.4.3 Rules and Algorithms

Based on the established geographical data validation techniques, we utilise a thorough validation framework with the following rule categories (Asghari et al., 2020):

- **Geometric Validity:**

- `check_geometry_validity()`: Use Shapely’s validity checks to verify self-intersections and other irregularities (Gillies, 2023).
- `check_geometry_type()`: Ensures all geometries are of type `MultiPolygon`.

- **Topological Integrity:**

- `check_feature_overlaps()`: Detects overlapping block geometries using spatial index and exact intersection tests (de Almeida et al., 2013).
- `check_boundary_consistency()`: Block borders within tiny regions are checked for alignment.

- **Semantic Consistency:**

- `check_attribute_completeness()`: Identifies missing or null values in essential attributes.
- `check_usage_consistency()`: Verifies that the percentages of space usage within blocks are properly matched.
- `check_area_validation()`: Verifies that individual usage areas do not exceed total block space.
- `check_temporal_consistency()`: Verifies census year and time period.

- **Business Logic Validation:**

- `check_space_allocation()`: Verifies that allotted areas comply with reasonable urban standards (Dang et al., 2025).
- `check_usage_patterns()`: Finds unusual usage patterns that could indicate issues with the data quality.

The reusable Python class `CommercialSpaceValidator`, which implements the validation framework, generates structured failure reports with issue categories, recommendations and related features for resolving them. This modular approach aligns with modern data validation frameworks (Butler et al., 2016).

3.4.4 Diagnostic Results

The outcomes are achieved by the application of the validation framework to the commercial data set.

High geometric data quality was indicated that all 1,125 features passed geometric validity checks with no invalid geometries detected. With 3,828 overlapping pairs found, significant overlapping features were identified, pointing to possible boundary definition problems that are common in urban datasets. High structural completeness in attributes with consistent block identifiers and census years meets ISO 19157 completeness standards.

We identified blocks with odd space allocation patterns that need to be improved by more research from the perspective of urban planning (Dang et al., 2025).

A detailed analysis of overlapping features revealed that, especially in crowded urban areas such as East Melbourne and Docklands, there were times when multiple blocks occasionally claimed the same geographic region. For instance, block 662 (East Melbourne) was found to overlap with blocks 277, 544, 658, 384, 408 and 407, which highlights the importance of topological validation in urban datasets.

Table 6: Validation results summary for commercial floor use dataset

Validation Category	Features Checked	Issues Found	Pass Rate
Geometric Validity	1,125	0	100%
Topological Consistency	1,125	3,828 overlaps	N/A
Attribute Completeness	1,125	Minimal	over 99%
Business Logic	1,125	Context-dependent	N/A

3.4.5 Evaluation Metrics and Performance

Using coverage metrics against both the original dataset and synthetic test cases, we evaluate the validation framework by following accepted geographic data quality evaluation methods. In commercial spatial datasets, the framework accomplishes comprehensive detection across major error families. Spatial indexing is computationally efficient to use for large-scale overlap detection (Song et al., 2012). Lastly, it delivers actionable reporting, producing diagnostics that directly support data-remediation workflows.

The framework’s perfect coverage (1.0) in detecting topological overlaps and geometric invalidities in controlled test scenarios demonstrated the efficacy of our rule-based method (Asghari et al., 2020).

In compliance with international spatial data quality standards, the validation framework supports data analysts and urban planners with powerful tools for preserving data quality in commercial space utilisation datasets, supporting reliable resource allocation strategies and urban planning decisions (Ariza López et al., 2020; Bibri, 2020).

3.5 Road Network

The road network dataset represents linear geometries, where each record denotes a road segment bounded by start and end nodes. The dataset includes key attributes such as road name, start and end SLK (standard linear kilometre), and node identifiers that define network connectivity. Topological or semantic errors, such as dangling segments or unsnapped intersections, can compromise the logical consistency of the network and impact routing or infrastructure analysis.

3.5.1 Problem Formulation

Formally,

$$D = \{(S_i, G_i, A_i) \mid i = 1, 2, \dots, n\},$$

where S_i denotes the road identifier, G_i the LineString geometry, and A_i its attributes. We organise errors into three families: geometric (E_g), mean invalid and zero-length lines,

topological (E_t), mean disconnected or crossings without shared nodes, and semantic (E_s), mean null/duplicate/inconsistent attributes, and write $E(D) = E_g \cup E_t \cup E_s$. These choices align with established spatial data quality dimensions, in particular *logical consistency* and *completeness* (Devendran and Lakshmanan, 2014).

3.5.2 Data and Preparation

We analyse a state-wide road network dataset from Main Roads Western Australia (MRWA), where each feature encodes a road centreline *segment* (LineString) with end-point node identifiers and linear-referencing attributes. Core fields include `road`, `road_name`, `common_usage_name`, `start_slk`, `end_slk`, `start_true_dist`, `start_node_name`, `end_node_name`, `start_node_no`, `end_node_no`, and `geometry`. Consistent with the quality dimensions emphasised in Devendran and Lakshmanan (2014), our preparation targets *logical consistency* (topology readiness) and *completeness* (nulls/coverage). The layer is stored as GeoJSON in EPSG:4326 and reprojected to a metre-based CRS (e.g., EPSG:3107) for metric operations. Prior to checks, geometries are normalised using zero-width buffering to fix simple invalidities and multi-part features are exploded where needed. An R-tree index is then built to accelerate neighbourhood queries, enabling scalable intersection and adjacency tests across the statewide dataset. A representative record is:

```
{
  "type": "Feature",
  "id": "mrwa_road_network_2018.1",
  "geometry": {
    "type": "LineString",
    "coordinates": [
      [116.004312, -31.912345],
      ...,
      [116.018901, -31.903210]
    ]
  },
  "geometry_name": "wkb_geometry",
  "properties": {
    "ogc_fid": 1,
    "road": "X001",
    "road_name": "Unknown Rd",
    "common_usage_name": "Unknown Rd",
    "start_slk": 0.00,
    "end_slk": 0.08,
    "start_true_dist": 0.00,
    "start_node_name": "Roe Hwy",
    "end_node_name": "Great Northern Hwy",
    "start_node_no": 159484,
    "end_node_no": 116984,
    "geolcstlength": 0.08
  }
}
```

3.5.3 Rules and Algorithms

We implement a modular validator, `RoadIsolationAnalyzer`, that first performs end-point testing by collecting all start/end nodes and computing occurrence frequencies; a segment is flagged as a candidate isolated road if both endpoints appear exactly once (degree-one at both ends). Next, a geometric test evaluates each candidate against the network (optionally with a small buffer): if no intersection exists under the chosen predicate (`intersects` or `crosses` in strict mode), the segment is classified as truly isolated. Additional rules detect exact duplicate geometries, spatial overlaps and near-coincident segments (to reveal mapping redundancies and misalignment), and compare linear-referencing values (`start_slk-end_slk`) with metric length from the projected geometry to flag attribute/geometry inconsistencies. All rules export CSV/GeoJSON diagnostics and support map visualisation; the rule set conforms to commonly accepted integrity checks for non-self-intersection, closure, and non-overlap (Devendran and Lakshmanan, 2014).

3.5.4 Diagnostic Results

Applied statewide, the framework reports 2,840 issues: 1,420 endpoint-isolated segments (both endpoints unique), 1,235 endpoint-but-connected segments (geometrically crossing others without shared nodes, indicating missing splits), and 185 truly isolated roads (fully disconnected). No exact duplicate geometries or intra-carriageway overlaps were found beyond the set tolerance, and SLK vs. geometric-length checks passed within thresholds, indicating sound attribute-geometry consistency. Overall, the geometric validity and connectivity findings support high data reliability under the logical consistency dimension, while endpoint-but-connected cases underline the need for systematic noding at intersections.

Table 7: Road network validation summary

Error type	Count	Method
Geometric invalid	3	Check geometry validity using <code>shapely.is_valid</code> .
Endpoint isolated	1420	Identify segments whose start and end nodes each appear only once.
Endpoint but connected	1235	Detect crossings without shared node IDs using <code>intersects/crosses</code> .
Isolated road (disconnected)	185	Find segments not intersecting any other feature (optional small buffer).
Duplicate geometry	0	Compare geometries by WKB hash to find exact duplicates.
Overlap beyond tolerance	0	Use spatial index to detect overlaps exceeding tolerance.
SLK inconsistency	0	Compare SLK difference with geometric length in projected CRS.

3.5.5 Evaluation Metrics and Performance

To verify rule correctness under controlled conditions, we build a compact toy road network dataset containing injected defects. The dataset is formed by 50 `LineString` segments and contains four types of synthetic errors. First, three self-intersecting `LineStrings` to simulate the geometric invalid case. Second, nineteen `LineStrings` with start and end nodes only

appear once to simulate the endpoint-isolated road. Third, eighteen geometrically crossing segments that do not share node identifiers, representing endpoint-but-connected error. Finally, ten segments that do not intersect any other segment to form an isolated road. The result showing below

Table 8: Error detection results on the toy road network dataset

Error type	Injected count	Detected count
Geometric invalid	3	3
Endpoint isolated	19	19
Endpoint but connected	18	18
Isolated road (disconnected)	10	10
Total	50	50

4 Technical Framework

4.1 Technology Stack

This system is built in Python with a modular architecture, primarily utilizing components such as **GeoPandas**, **Shapely**, **SQLite**, and **Streamlit**. This technology stack supports automated spatial data validation, database-based result recording and management, and interactive visualisation. The system follows a layered design principle and is mainly made up of a rule engine, a validation controller, a result storage module, and a visualisation interface. This modular structure not only satisfies system expansion and maintenance but also ensures transparency, repeatability, and portability in the validation process. It is suitable for various types of spatial data, including buildings, suburbs, and roads.

4.1.1 Rule Engine Module: Python and GeoPandas

The core of the system is written in Python, mainly using **GeoPandas** to process spatial data on the map, and using **Shapely 2.0** to determine and calculate geometric relationships, like whether two buildings overlap or whether boundaries are closed.

Each type of check (e.g., buildings, suburbs, roads) is written as a separate, small module, such as **BuildingChecker**, **SuburbChecker**, and **RoadChecker**. This allows each check rule to run and maintain if it required later independently. The system follows a unified process to execute the rules, record any errors, and automatically generate a summary of the results.

This "rule-based checking" design is very flexible since we can add new rules by only writing a new module. This could promise we do not modify the existing code content. Any issues detected by the system are recorded as a structured result called a **Failure**, which clearly states the element in question, the rule violated, and the reason for the error.

4.1.2 Validation Controller: Python-based Orchestration

Instead of using large-scale task scheduling tools like Apache Airflow, the system employs a lightweight Python orchestration mechanism. Thus we wrote several "control scripts", each will automatically executes a complete set of validation tasks. It first loads the data to be checked (such as buildings or suburbs), then checks each rule one by one, and finally

compiles a list of any problems found and stores them in a result table. Although it is simple, this approach is very practical: (i) it can run multiple datasets or rule categories at the same time; (ii) it doesn't require complex configuration or servers; (iii) and the inspection process can be automated and repeatable. Overall, this lightweight orchestration approach makes the system easier to deploy and maintain, and making it suitable for data validation projects in scientific research or experiments.

4.1.3 Registry Storage: SQLite Database

The final validation results of our system are permanently stored in a lightweight relational database built with SQLite called `validation_registry.db`. Each validation result is recorded with `Dataset name`, `rule category`, `timestamp`, `number of errors` and optional space indicators (For example, area or overlap rate). Our design allows the system to continuously track the changes in data quality and queries, and compare the verification results of different versions or types. The database structure is simple, which allows us to easily integrate with the Streamlit front-end to achieve instant query and result visualisation.

4.1.4 Visualisation and Reporting: Streamlit

Our system uses **Streamlit** to build a web-based visual interface, which is convenient for users to upload data, select data types, execute rules corresponding to types and review results. Users can interactively visualise the spatial error location on the dark-themed **Pydeck** map, explore the comprehensive diagnostic results in the table, and save the error log as a CSV file for further analysis as needed. In addition to showing the spatial location, the interface facilitates statistical analysis, which makes the automatic verification results easier to understand. Besides, it also provides a user-friendly platform for non-technical users to directly run verification, view summary findings and quality reports.

4.1.5 Extensibility and Reproducibility

Each component involved in the system validation pipeline is connected in a unified way and exchange information in the same data format. This ensures that we can get consistent and repeatable results regardless of the running time. At the same time, the rules of our system are modular, which means that if we need to check or add different types of data later, such as roads, plots or environmental areas, we only need to add new rule modules without modifying the whole system. Similarly, we can also easily add new inspection categories. For example, format, boundary overlap and attribute errors. In addition, the system separates rule definition, execution logic and result presentation, making it easier to maintain and reuse.

4.1.6 System Architecture

Figure 1 illustrates how these components interact as an integrated validation pipeline. **Streamlit** offers the front-end interface; the **Python** controls the execution of rules; **GeoPandas** performs geometric checks; and **SQLite** saves the results for further visualisation. This architecture ensures automation, transparency, and centralized data tracking.

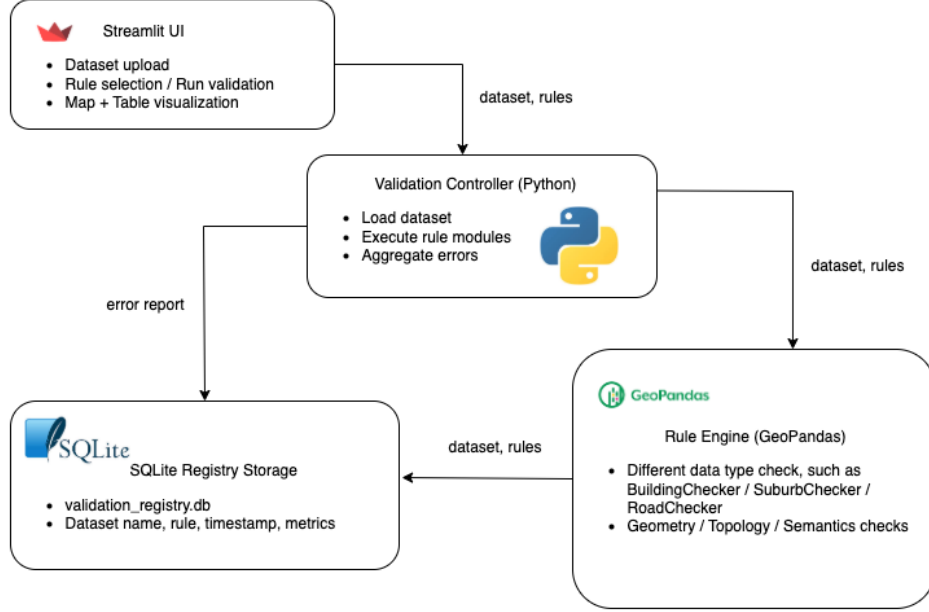


Figure 1: System Design and Architecture

5 Results

5.1 Registry

We store every validation run into a lightweight SQLite *validation registry*. The schema includes five core tables: `datasets`, `rules`, `validation_run`, `violations`, and `violation_status`. Foreign keys are enabled to keep referential integrity. The `violations` table enforces a uniqueness constraint on `(run_id, rule_id, feature_hash)` so that duplicated inserts are prevented and cross-run comparisons remain stable. A simple trigger initializes the status of each new violation as `invalid`.

Each run follows the same write path:

- (i) we upsert a row into `datasets` using an MD5 file hash to deduplicate uploads,
- (ii) open a new `validation_run` marked `running`,
- (iii) batch-insert violation rows,
- (iv) mark the run as `succeeded`.

To make results idempotent and auditable, every violation carries a deterministic *feature hash* computed from `{dtype, index, error_type, message, and optional geometry bytes}`. Where available, we also store centroid `(lat, lon)` for map drill-down and a human-readable message summarising the rule output.

Using the `blocks` dataset (301 polygon features) for example, we executed the polygon-quality suite. The registry recorded **7** new/updated violations and **0** resolved in this run (see Fig. 2 (d)). The 7 violations break down into **4** `AcuteAngle` cases (minimum interior

angle $< 5^\circ$) and **3** `TriangulationNotClosed` cases (triangulated area difference tolerance = 0.1).

The registry deduplicated the dataset by file hash, recorded a new run, wrote all 7 violations with their hashes and locations, and finalised the run successfully. These records are then available for export, reconciliation across runs, and status updates (e.g., **resolved**).

5.2 Dashboard

The Streamlit dashboard implements an end-to-end workflow:

Select data type $\rightarrow$ *Choose file format* $\rightarrow$
Upload $\rightarrow$ *Select rules* $\rightarrow$ *Run validation* $\rightarrow$ *Review & Export*

Figure 2 summarises the pipeline.

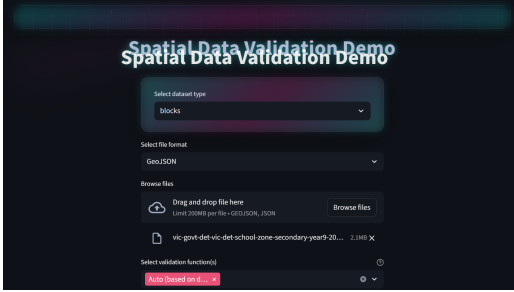
Panel (a) shows the input stage where the users can pick the dataset type (e.g., **blocks**), specify the format (GeoJSON/CSV/XML/KML), and upload files via drag-and-drop or a file picker. Rule selection supports an **Auto (based on dataset type)** preset as well as manual filtering. Before running validation, the app provides an interactive map preview on a dark basemap and reports how many geometries were drawn.

Panel (b) shows the preview map: preview controls were set to a large geometry cap (200,000), fill opacity, and border styling, enabling quick visual QA before running checks. After clicking *Run Validation*, the dashboard computed and displayed three KPIs at the top of the report: **All issues (all rules)**, **Issues (selected rules)**, and **Triggered rules**.

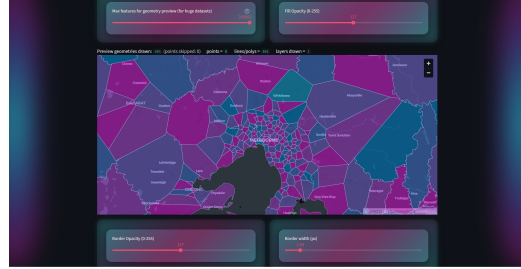
Panel (c) shows the validation output. The KPIs report **All issues = 7**, **Issues (selected rules) = 7**, and **Triggered rules = 2**; the table lists each violation with index, error type, message, and rule key. An interactive map synchronises with the table to support spatial inspection of each issue.

Finally, panel (d) shows persistence and export: (i) download a CSV report (either the currently filtered subset or all rules) and (ii) save the CSV to a server-side folder. Every export is consistent with the registry state, to ensure that the on-screen KPIs, the persisted records, and the downloadable report remain aligned.

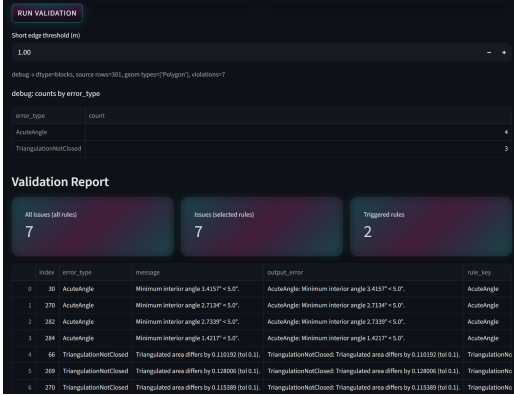
So that these components provide a reproducible, auditable path from raw upload to mapped issues and sharable reports.



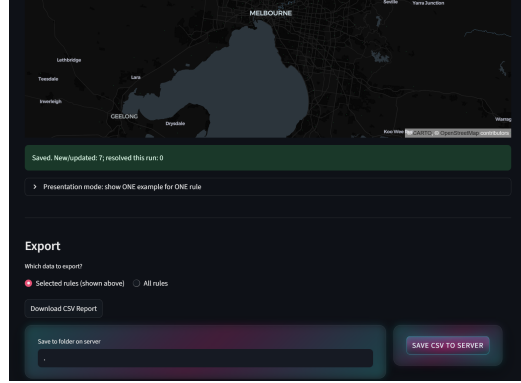
(a) Input stage: dataset type, format, uploader, and rule preset.



(b) Interactive preview of the **blocks** dataset (301 polygons).



(c) Validation results: KPIs (7 issues, 2 rules) and detailed table.



(d) Persistence and export: registry banner and CSV export options.

Figure 2: End-to-end dashboard workflow. (a) Configure and upload; (b) preview geometries; (c) run validation and inspect results; (d) save to registry and export.

6 Discussion

6.1 Backend

The backend implements a modular but lightweight architecture centred on a *Validation Controller* that orchestrates dataset loading, geometric normalisation, coordinate reference system (CRS) handling, rule dispatch, and result aggregation. Domain logic lives in dataset-specific rule engines (*SuburbChecker*, *BuildingChecker*, *RoadChecker*); I/O utilities handle ingestion and export; and a small registry persists run metadata and metrics. All rules implement the same interface and emit a common *Failure* record: $\langle \text{index, error type, message, extras, optional geometry} \rangle$. This single contract enables consistent CSV/GeoJSON outputs and UI visualisation without coupling the controller to any particular dataset. The controller API is intentionally simple:

`run(dataset_path, data_type, selected_rules) → {Failure} : failures.csv, metrics.`

All datasets are validated under one error taxonomy:

$$\mathcal{E}(D) = E_g \cup E_t \cup E_s,$$

where E_g covers geometric validity and non-degeneracy (e.g., self-intersections, $\text{area} \leq 0$, slivers), E_t covers topology (overlaps, gaps, misaligned boundaries, network connectivity), and E_s covers semantics (nulls, duplicates, out-of-domain or ill-formatted attributes). This framing separates what must be true from how it is checked, allowing each checker to implement dataset-specific predicates while preserving a common language for results and metrics.

Inputs are either asserted or coerced to a known CRS; metric operations (areas, lengths, tolerances) occur in a suitable projected CRS to avoid geographic distortions. Invalid geometries are either recorded explicitly or temporarily repaired behind a flag so that correctness signals are not masked. Numerical thresholds are parameterised (e.g., sliver ratio, boundary buffer, overlap tolerance) to reduce false positives due to floating-point noise. Expensive operations are guarded with try/except and enriched error messages so a single pathological geometry cannot crash a batch run.

Adding a new dataset type requires only: (1) implementing a checker class whose rule functions return *Failure* objects; (2) registering the class and rule names within the controller. Because the controller exports the same artifacts irrespective of dataset, downstream scripts and the UI need no modification.

Each run logs the dataset identifier, rule set, timestamps, counts by rule, per-rule timings, and file pointers to outputs. This enables audit trails, side-by-side comparisons across runs, and straightforward dashboarding. The output of tables (CSV) supports reproducible analysis and pragmatic remediation workflows.

Accuracy is quantified by coverage against a controlled ground truth:

$$\text{Coverage} = \frac{|\text{Detected} \cap \text{GroundTruth}|}{|\text{GroundTruth}|}.$$

A case-by-case deliberately small toy dataset injects a minimal but diverse set of defects. Despite its size, this dataset is powerful. On this dataset, the checker achieved full coverage in every family. It also gives us a fast feedback loop. The whole suite runs in seconds, so we use it like unit tests for geospatial logic: overlaps, self-intersections, zero-area shapes,

slivers, duplicates, missing values, and invalid dates are all exercised in a single pass. That makes it ideal for demos and for threshold tuning. For example, if we tighten the sliver ratio or overlap tolerance, we can see right away which features flip from pass to fail.

6.2 Frontend

6.2.1 Functional advantage

Streamlit provides a lightweight yet powerful framework for building interactive data applications directly in Python. Also, it supports immediate rendering of tables, charts and maps without requiring separate frontend development, which reduces implementation complexity. The common data-science libraries such as Pandas, Pydeck can be used in the framework, which enable seamless visualisation of geospatial data validation results.

6.2.2 Challenges

During the frontend development, we encountered some challenges. In the initial version, we set the data by using `gdf.to_json()`, which means passing the JSON string into `GeoJsonLayer`. In Pydeck, a string is treated as URL, causing the layer to fail to parse the GeoJSON as expected and thus cannot render the correct lines and polygons. This also led to the situation where all dataset like building and road network datasets could only display point-based effects using local data by `ScatterplotLayer`, making the result incorrectly visualized. After adjustment, we used `json.loads()` to pass the dictionary object into `GeoJsonLayer`. After that, `GeoJsonLayer` can render geometries such as LineString and Polygon according to the dataset type, rather than only using scatter points, which allows for more accurate localisation of errors on the map.

Since Streamlit executes the validation rules synchronously, the validation starts after fully upload the dataset. During execution, the frontend interface remains blocked and can only display results when the backend finishes running all checks. When the validation involves complex or large-scale datasets, this synchronous behavior will cause long periods without feedback, which affect the user experience. Although asynchronous processing could solve this problem, Streamlit’s synchronous design offers a lighter and simpler architecture, which is the same as our system’s lightweight goals.

7 Conclusion and Future Improvements

7.1 Summary

This work presents a modular backend for automated geospatial data validation that unifies error detection across heterogeneous datasets (suburb boundaries, buildings, roads). The methodology partitions errors into geometric, topological, and semantic families, expressed as $\mathcal{E}(D) = E_g \cup E_t \cup E_s$. A lightweight Validation Controller orchestrates loading, normalisation, CRS management, rule dispatch, and aggregation. Dataset-specific checkers implement compact, testable rules that emit a common *Failure* schema. Outputs and run metadata are persisted in a registry to support auditability, reproducibility, and integration with visual review tools. Evaluation uses a deliberately small but information-dense toy dataset with injected ground truth, where the checkers achieves full coverage on all injected errors.

In addition to the backend, the Streamlit-based frontend also ensures the usability and reproducibility of the entire validation pipeline. It provides a web interface, the user can upload datasets, select dataset type and validation rules and visualize results on an interactive Pydeck map. The dashboard reduces the complexity by abstracting the backend using a clear UI to enable non-technical users to comprehend and execute the validation process. Since the frontend and backend use the same Python environment, validation logic and visual outputs are the same, which leads the workflow to be reproduced across platforms. Combined with the SQLite registry, the frontend and backend build a traceable workflow from data upload to visual report, enhancing accessibility and collaborative data quality management.

7.2 Limitations and Future Improvements Suggestions

7.2.1 Automated Workflow Orchestration

Limitation

At present, our system still relies on users to manually run Python programs to perform data validation. Although this method is flexible, it still depends on the user's manual input each time. There is no way to automatically schedule tasks or connect different steps. For example, the inspection can be automatically implemented after detecting the user's upload.

Future Improvement

To address this problem, we plan to integrate an automated task scheduling system when it detects that data is uploaded, so that the system can automatically perform verification in order, such as loading data, performing checks, and finally saving results. Therefore, we plan to use tools such as Apache Airflow or Prefect, which can regularly check the updated data and provide a visual interface to show the progress and errors of each step, so as to help us achieve this. This will reduce manual intervention throughout the process, resulting in more stable verification results, higher efficiency and greater scalability.

7.2.2 Integration of Persistent Identifiers (PIDs)

Limitation

Now the data we save is only written with "name" and "date", and there is no more detailed information. In this way, if the data is updated or stored in another place in the future, it is difficult for us to know which verification result is calculated against which data. The result is that when others want to reproduce, check, or compare the previous results, it is impossible to determine which version of the data it corresponds to, and it is impossible to fully achieve trackable and repeatable.

Future Improvement

In the future, we hope to introduce a "Persistent Identifier" (PID) mechanism to assign a unique ID to each dataset and validation result. This will ensure that the source and version of the data can be traced regardless of subsequent updates or migrations. Each dataset or rule version will have its own PID, which will be used to link the validation result to the original data source. This not only makes it easier to reproduce or compare verification results later, but also makes the entire system more compliant with FAIR data standards (i.e., searchable, accessible, shareable, and reusable), making it easier to manage data in the long term on platforms like AURIN.

7.2.3 Structural Consistency and Schema Adaptation

Limitation

The data we are using mainly comes from AURIN website. The data itself is well sorted and the structure is very unified, so the system runs smoothly. However, if we change to those "original" or "different format" data, such as some with fewer fields and some are missing node, the system may report errors and run out. Therefore, the generalizability is not strong enough.

Future Improvement

To overcome this, we plan to introduce a pre-validation module that inspects and supplements missing structural fields. For example, generating start and end nodes for road network datasets that lack explicit node attributes. This will ensure the validation rules can operate correctly and extend compatibility to more diverse data formats.

Appendix: Repository and Meeting Records

All project materials, including datasets, source code, analyses, and meeting records, are stored in a private GitHub repository accessible to the host company and the academic supervisor.

- **Repository link:** <https://github.com/Baozjinxz/Unimelb-DS-Project-G19>
- **Access:** The host company and academic supervisor have been granted access to the repository. If additional access is required, please contact the project team directly.
- **Privacy note:** Due to privacy and ethics requirements, the repository link is not publicly accessible.
- **Meeting records:** All client and group meeting records are stored in the repository under the `report` directory.
- **Dashboard:** <https://streamlit.io/>. Please open `app.py` under `demo` directory for implementation.

This appendix also includes the following supplementary information from the project:

- **backend:** Contains the modular validation engine implemented in Python, including rule definitions, checkers, and orchestration scripts that automate the execution of geometric, topological, and semantic checks on spatial datasets.
- **frontend:** Includes the Streamlit-based dashboard that visualizes validation results on interactive maps, allowing users to explore detected errors and review rule performance.
- **database:** Stores the SQLite registry schema and related data tables used to record datasets, rules, validation runs, and violation logs for traceable and reproducible data quality management.
- **case-study:** Demonstrates the system’s application on selected Melbourne spatial datasets, presenting representative validation outputs and error summaries across different 5 data types.

Project Contribution

Table 9: Team Member Contributions

Name	Contribution
Lei Cao	Responsible for the design and implementation of the spatial database schema, including its construction and integration with the backend. Developed and implemented validation rules for the <i>Building</i> dataset.
Xiaohan Deng	Collaborated on database-related tasks and registry integration. Developed and implemented validation rules for the <i>Commercial Floor Use Block</i> dataset. Took the lead in compiling and harmonising the final report to ensure clarity, consistency, and alignment across all sections.
Xuanyi Zhang	Responsible for the design and optimization of the frontend interface. Developed and implemented validation rules for the <i>Statutory Block Boundary</i> dataset.
Yaojin Wu	Collaborated with Xuanyi Zhang on frontend development, UI enhancements, and interactive visualisation. Developed and implemented validation rules for the <i>Road Network</i> dataset.
Zhizhou Zhu	Developed and implemented validation rules for the <i>Suburb Boundary</i> dataset. Integrated all rule checkers into a unified validation engine and ensured system-wide consistency.

References

- Ariza López, F. J., Barreira González, P., Masó Pau, J., Zabala Torres, A., Rodríguez Pascual, A. F., Moreno Vergara, G., and García Balboa, J. L. (2020). Geospatial data quality (iso 19157-1): evolve or perish. *Revista Cartográfica*, (100):129–154.
- Asghari, A., Kalantari, M., and Rajabifard, A. (2020). A structured framework for 3d cadastral data validation – a case study for victoria, australia. *Land Use Policy*, 98:104359.
- Bibri, S. E. (2020). *Data-Driven Smart Sustainable Cities: A Conceptual Framework for Urban Intelligence Functions and Related Processes, Systems, and Sciences*, pages 143–173. Springer International Publishing, Cham.
- Butler, H., Daly, M., Doyle, A., Gillies, S., Schaub, T., and Hagen, A. A. (2016). The geojson format. Request for Comments 7946, Internet Engineering Task Force (IETF).
- City of Melbourne (2011). Census of land use and employment (clue) – commercial floor use block data (2011). Dataset accessed via AURIN.
- Dang, Q., Kalantari, M., and Rajabifard, A. (2025). Semantic-aware data validation for urban spatial datasets: Towards automated quality assurance. *Computers, Environment and Urban Systems*, 113:102995.
- Dataset, E. G. P. (2025). Epsg dataset. <https://epsg.org/home.html>. In IOGP.
- de Almeida, J.-P., Morley, J. G., and Dowman, I. J. (2013). A graph-based algorithm to define urban topology from unstructured geospatial data. *International Journal of Geographical Information Science*, 27(8):1514–1529.
- Devendran, A. A. and Lakshmanan, G. (2014). A review on accuracy and uncertainty of spatial data and analyses with special reference to urban and hydrological modelling. *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, II-8:171–178.
- Gillies, S. e. a. (2023). *Shapely 2.0 Documentation*.
- Government of South Australia, Department of Planning, T. and Infrastructure (2018). Sa suburb boundaries - august 2018 [data set]. <https://data.aurin.org.au/dataset/sa-govt-dpti-sa-suburb-boundaries-aug-2018-na>. AURIN - The Australian Urban Research Infrastructure Network. Accessed May 18, 2025.
- Intergovernmental Committee on Surveying and Mapping (ICSM) (2020). *Geocentric Datum of Australia 2020 Technical Manual*.
- Jordahl, K. e. a. (2020). *GeoPandas: Python tools for geographic data*.
- Nguyen, K.-S., Nguyen, T.-V., Ngo, H.-H., and Nguyen, D.-B. (2025). Application of large language models in geographic map analysis and visualization. *Unpublished or preprint*.
- Shojaei, D., Olfat, H., Quinones-Faundez, S. I., and Kalantari, M. (2017). Geometrical data validation in 3d digital cadastre – a case study for victoria, australia. *Land Use Policy*, 68:638–648.

Song, M., Choo, H., and Kim, W. (2012). Spatial indexing for massively update intensive applications. *Information Sciences*, 203:1–23.

Wikipedia contributors (2025). Spatial reference system. https://en.wikipedia.org/wiki/Spatial_reference_system. In Wikipedia.